



Agora: Toward Autonomous Bug Detection in Production-Level Consensus Protocols with LLM Agents

Xiang Liu^{*1} Sa Song^{*2} Zhaowei Zhang³ Huiying Lan¹ Jason Zeng⁴ Ming Wu⁴ Michael Heinrich⁴
Yong Sun² Ceyao Zhang³

Abstract

Consensus protocols form the backbone of distributed systems and blockchains, where implementation bugs can cause data corruption and financial losses. While LLM-based approaches show promise in code analysis, they struggle with deep protocol-level logic bugs involving complex state-dependent behaviors across multiple execution stages. We present Agora, a domain-aware multi-agent framework that integrates hypothesis-driven testing with LLM capabilities for systematic protocol verification. Agora employs specialized agents that collaboratively explore protocol state spaces, synthesize attack scenarios using domain-specific constraints, and validate findings through iterative refinement. This explicit role separation enables reasoning about global protocol invariants beyond single-function code analysis. We evaluate Agora on four consensus implementations (Raft, EPaxos, HotStuff, BullShark) using four state-of-the-art LLMs. Agora discovers 15 previously unknown protocol-level logic bugs that violate safety properties, while existing LLM-based agents fail to detect any such protocol-level logic bugs. Our results demonstrate that domain-aware multi-agent collaboration is essential for detecting deep logic bugs in complex protocols.

1. Introduction

Consensus protocols, including Crash Fault-Tolerant (CFT) (Lamport, 1998; 2001; Moraru et al., 2013; Ongaro & Ousterhout, 2014) and Byzantine Fault-Tolerant (BFT) (Cas-

tro, 1999; Lamport, 2011; Yin et al., 2018; Danezis et al., 2021) protocols, serve as the cornerstone of modern distributed systems, ensuring consistency and availability in the presence of faults. Among these, the CFT protocols are widely adopted in distributed databases (Taft et al., 2020), storage systems (Corbett et al., 2013), and cluster coordination services (Hunt et al., 2010) to guarantee the correctness of online transactions and data storage. Conversely, the BFT protocols form the backbone of blockchain systems (Nakamoto, 2008), designed to secure transactions against malicious actors.

Consequently, any defects in the implementation of these consensus protocols can lead to severe consequences, ranging from data corruption to significant financial losses (Zheng et al., 2017). Despite the critical importance of these systems, bug detection for distributed protocol implementations remains a formidable challenge and has long been a focal point of research. The inherent complexity of distributed systems, coupled with the intricate logic of consensus algorithms, makes subtle bugs notoriously difficult to detect (Ongaro & Ousterhout, 2014). Even widely used and mature libraries (e.g., etcd (etcd-io, 2026)), which have been under development for many years, are still found to contain critical bugs that require experienced engineers extensive periods to uncover and resolve (Zhou & et al., 2021).

In recent years, the rapid advancement of Large Language Models (LLMs) (Achiam et al., 2023; DeepSeek-AI et al., 2025) and autonomous agent systems (Yao et al., 2023; Cheng et al., 2024) has revolutionized software engineering tasks, helping engineers save significant time and rapidly perform tasks such as code analysis (Feng, 2020) and bug detection (Chen et al., 2021). While LLMs and agents have demonstrated strong capabilities in identifying common coding errors and vulnerabilities (Kaniewski et al., 2025), their effectiveness remains limited: they primarily excel at detecting superficial or localized bugs, but struggle to reason about the complex, state dependent logic inherent in distributed consensus protocols (Wang et al., 2024c;b). Therefore, to fully leverage the general code understanding capabilities of LLMs and efficiently detect bugs in consensus protocols and their implementations, it is necessary to

^{*}Equal contribution ¹School of Computing, National University of Singapore ²School of Information and Telecommunication Engineering, Beijing University of Posts and Telecommunications ³Peking University ⁴OG Labs. Correspondence to: Yong Sun <sun-yong@bupt.edu.cn>, Ceyao Zhang <ceyaozhang@pku.edu.cn>.

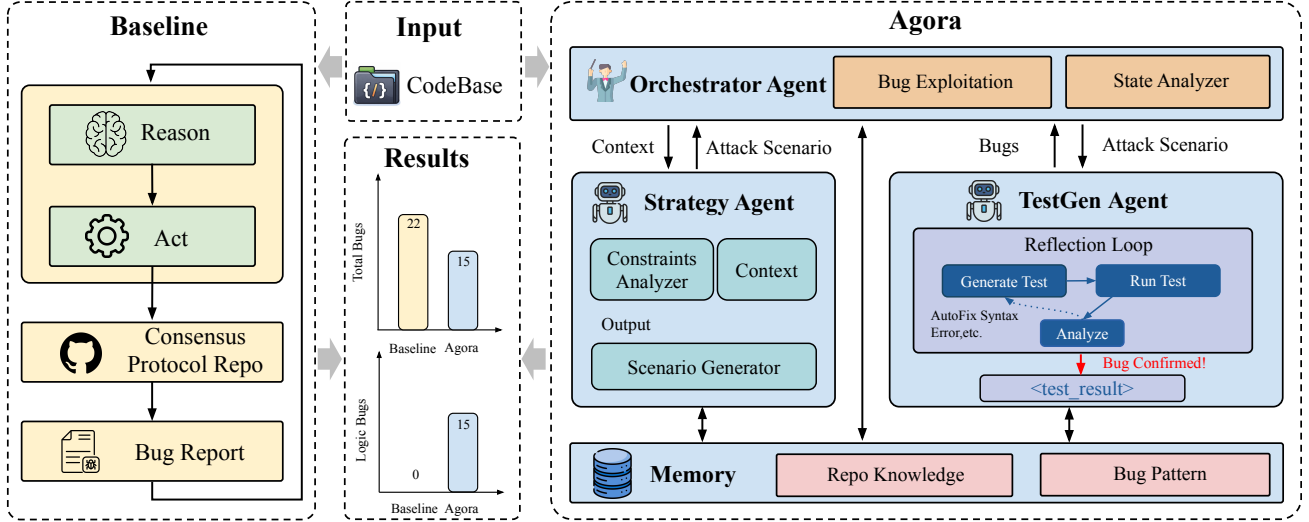


Figure 1. **Left** is the baseline overview. **Middle** shows the input and result comparison where baselines could only detect **low-level implementation bugs**, but Agora could detect critical **protocol-level logic bugs**. **Right** is the overview of Agora, which consists of three agents: Orchestrator agent, Strategy agent and TestGen agent.

integrate domain knowledge of distributed consensus with multi-agent systems (Yang et al., 2025b;a). Such a fusion enables the identification of subtle errors that may otherwise lead to severe data inconsistencies or substantial economic losses, thereby ensuring system correctness and security for researchers and engineers.

Therefore, in this paper, we first analyze bugs in consensus protocols and identify corresponding patterns, and summarize the constraints under which bugs occur in consensus protocols. Then, based on the testing principle of hypothesis-driven testing (HDT) (Kell & Oliver, 2004), we are the first to construct a multi-agent system Agora to better integrate domain knowledge with the strong code understanding and generalization capabilities of LLMs to analyze consensus protocol code repositories and efficiently discover bugs in consensus protocols. Specifically, as shown in Figure 1, our Agora is composed of three agents to realize the decoupling of workflow control, consensus scenarios properties, and test generation, enabling efficient deep bug hunting in consensus protocols: (1) **Orchestrator** agent is responsible for agent interactions and summarizing existing bugs, (2) **Strategy** agent is responsible for generating special consensus protocol bug scenarios, (3) **TestGen** agent generates corresponding unit tests based on the bug scenarios to expose bugs. We also used our Agora system to conduct extensive experiments on the repositories of four popular and widely used research- and production-level consensus protocols to validate the effectiveness of our approach. Our main contributions are summarized as follows:

- We are the first to combine domain knowledge of distributed consensus with LLM-based agents to detect

the protocol-level logic bugs (§3.1).

- We implement a flexible agent framework, Agora, which effectively follows the popular hypothesis-driven testing (HDT) paradigm and employs three agents (Orchestrator, Strategy, and TestGen) to automatically perform code bug detections for multiple consensus protocol code repositories (§3.2).
- We evaluate Agora using multiple LLM models and conduct extensive experiments on four popular consensus protocol repositories implemented in different programming languages. Our approach discovers 15 zero-day bugs, demonstrating that Agora can significantly improve the safety of distributed systems (§4.2).
- We conduct comparative analysis and systematic ablation studies, demonstrating that domain-aware multi-agent collaboration is essential: removing a single component reduces effectiveness by 73–100%, and each design choice contributes critically to performance (§4.3).

2. Preliminary and Related Works

2.1. Hypothesis-Driven Testing

Compared with traditional testing, which only focuses on verifying software does what it is supposed to do, hypothesis-driven testing (HDT) (Kell & Oliver, 2004) aims to explore under what conditions the software may fail. By explicitly formulating vulnerability hypotheses, HDT enables higher code coverage and more systematic bug discovery. A vulnerability hypothesis is defined as a tuple $H = (C, A, E, O)$, where C denotes the preconditions required to trigger a vulnerability, A represents the action se-

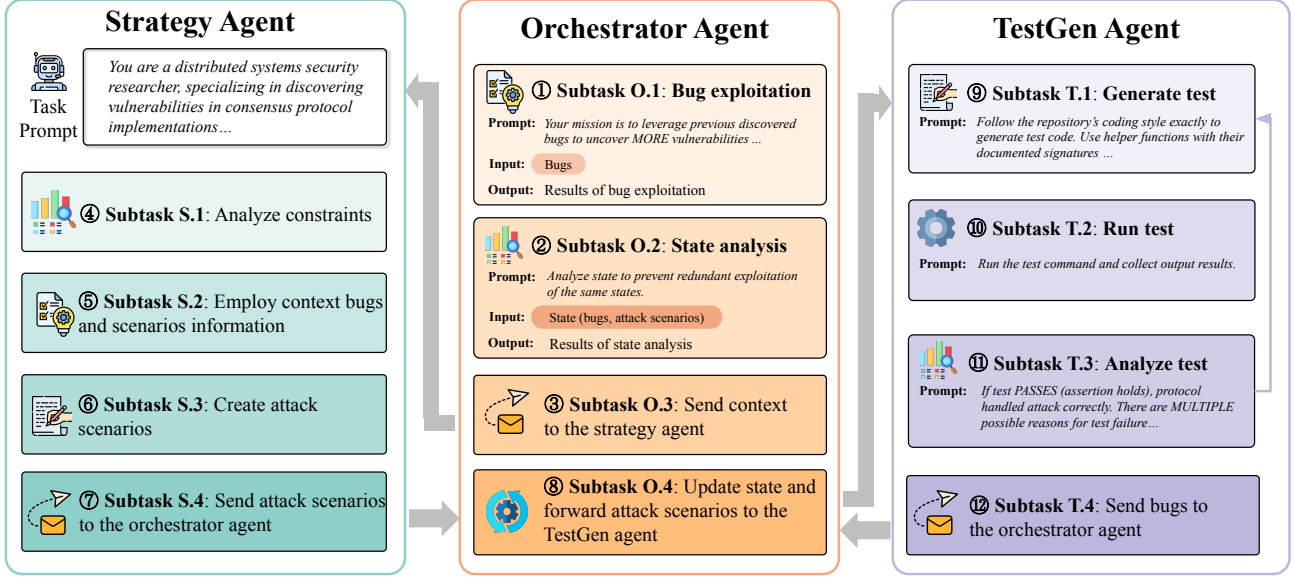


Figure 2. The overview of Agora workflow, which consists of three agents and twelve steps/subtasks (following the time order, from Step ① to Step ⑫). **Orchestrator** agent is responsible for bug exploitation, coordinating the agents, maintaining global state, and sending and receiving messages among agents; **Strategy** agent is responsible for generating attack scenarios based on the analysis results and the information it receives; **TestGen** agent is responsible for generating valid unit tests through a reflection loop to detect bugs in consensus systems. Each agent consists of four subtasks.

quence that may activate it, E specifies the expected buggy behavior, and O defines the oracle assertions used to validate the bug. Therefore, the goal of Agora is to construct an automated loop that follows the HDT paradigm and continuously discovers bugs by generating, instantiating, and validating vulnerability hypotheses.

2.2. Consensus Protocols

Consensus protocols mainly fall into two categories: Crash Fault-Tolerant (CFT) protocols, such as Paxos (Lamport, 1998; 2001), Fast Paxos (Lamport, 2006), EPaxos (Moraru et al., 2013), and Raft (Ongaro & Ousterhout, 2014); and Byzantine Fault-Tolerant (BFT) protocols, such as PBFT (Castro, 1999), HotStuff (Yin et al., 2018), DAG-Rider (Keidar et al., 2021), Narwhal-Tusk (Danezis et al., 2021), and BullShark (Giridharan et al., 2022). CFT consensus protocols are primarily used in traditional banking systems and Internet companies for data transactions and cluster coordination, whereas BFT consensus protocols form the foundation of blockchain systems and on-chain transactions. These protocols are widely used in both research and production settings, with code repositories implemented in multiple programming languages, such as Go and Rust. The distinction is particularly important because BFT systems explicitly tolerate malicious behavior from participating nodes, whereas CFT systems do not; as a result, analyzing these two classes of protocols involves fundamentally different **Constraints**.

To analyze bugs in consensus protocols, it is essential to summarize **bug patterns** previously identified through extensive human expertise, in order to more effectively uncover bugs that violate system safety and correctness and may lead to erroneous transactions and data corruption (i.e., protocol-level **Logic Bugs**), rather than low-level **Implementation Bugs** such as out-of-memory errors, memory leaks, integer overflows, or simple logical flaws.

The distributed protocols are primarily required to satisfy two fundamental properties: liveness and safety. Only the causes that lead to violations of these two properties constitute true protocol-level logic bugs. Such causes can be broadly categorized into five classes: (i) Recovery&Execution Divergence, (ii) Persistence&Monotonicity Violations, (iii) Dependency&Topology Flaws, (iv) Message Binding&Signature Violations, and (v) Resource&Operational Visibility Violations.

2.3. LLM-based Multi-Agent System

The paradigm of LLM applications focuses on multi-agent architectures, which decompose complex tasks into collaborative, specialized roles. In this way, multi-agent systems enable more focused reasoning, mitigate hallucinations, and manage long-horizon objectives more effectively than single-agent models (Chen et al., 2023; Guo et al., 2024; Yang et al., 2025c). The efficacy of this paradigm also enables structured coordination mechanisms and communication protocols (see the formalization in Appendix B).

Algorithm 1 Agora Workflow

```

1: Input: Repository Knowledge  $\mathcal{R}$ , Bug Patterns  $\mathcal{P}$ , Protocol Constraints  $Con$ 
2: Output: Detected Bugs  $\mathcal{B}$ 
3:  $\mathcal{S} \leftarrow \emptyset$  {Initialize global state}
4: while exploration budget remains do
5:   // — Orchestrator Agent —
6:    $\mathcal{BE} \leftarrow \text{bug-exploitation}(\mathcal{B})$  {Analyze historical bugs found}
7:    $Z \leftarrow \text{state-analyzer}(\mathcal{S})$  {Summarize global state}
8:   // — Strategy Agent —
9:    $AttS \leftarrow \text{Strategy}(\mathcal{BE}, Z, Con, \mathcal{P}, \mathcal{R})$  {Synthesize attack scenarios}
10:  SendToOrchestrator( $AttS$ ) {Update  $\mathcal{S}$ }
11:  // — TestGen Agent —
12:  repeat
13:     $T \leftarrow \text{GenerateUnitTests}(AttS, \mathcal{R})$  {Repo knowledge-based gen unit test}
14:     $R \leftarrow \text{ExecuteAndAnalyze}(T)$ 
15:  until  $R = \text{Success} \vee \text{MaxRetries}$ 
16:  if  $R = \text{Success}$  then
17:    SendToOrchestrator( $T$ ) {Update  $\mathcal{S}$ }
18:  end if
19: end while

```

Early dual-agent role-playing systems, such as CAMEL (Li et al., 2023) and ProAgent (Zhang et al., 2023), demonstrate that collaboration among agents can elicit step-by-step reasoning. More recent frameworks extend this idea by assigning explicit roles to multiple agents: MetaGPT (Hong et al., 2023) and ChatDev (Qian et al., 2023) employ manager, designer, and coder agents to simulate software engineering workflows; Magnetic-One (Fourney et al., 2024) and AG2 (Wang et al., 2024a) introduce a central coordinator that assigns tasks to different workers. OWL (Hu et al., 2025) further decouples multi-agent systems, enabling effective transfer across domains to solve new tasks. However, existing multi-agent systems still lack the ability to reason deeply about code patterns, especially to integrate domain knowledge of consensus protocol systems for efficiently discovering deep bugs.

2.4. LLM-powered Bug Detection

Recent advances in LLMs have significantly influenced automated bug detection and program analysis. Prior work has demonstrated that LLMs can assist developers in tasks such as bug localization, program repair, test generation, and vulnerability detection by leveraging their strong code understanding and natural language reasoning capabilities (Feng, 2020; Chen et al., 2021; Pearce et al., 2022).

LLMSAN (Wang et al., 2024c), SCALE (Wen et al., 2024), LLMDFA (Wang et al., 2024b) and PrimeVul (Ding

et al., 2025) all leverage LLMs to assist in bug detection. KNighter (Yang et al., 2025a) is the first to employ an agent-based system for analyzing bugs in LLM system code. However, these existing LLM-powered bug detection techniques largely focus on localized code patterns (**implementation bugs**) and lack the ability to reason about complex, state-dependent behaviors that span multiple components or execution stages (**logic bugs**). Moreover, these approaches do not leverage multi-agent systems to combine with HDT paradigm to achieve more comprehensive bug discovery.

3. Method

3.1. Motivation and Design Goal

We introduce Agora, which integrates domain knowledge of distributed consensus with an LLM-based multi-agent system to address the fundamental challenge of comprehensively detecting protocol-level logic bugs through HDT. Beyond this core capability, Agora is designed with several desirable properties. It is **effective**, capable of uncovering deep logic bugs; it provides **interpretability**, producing detailed analysis reports for discovered bugs to facilitate debugging; it is **efficient**, requiring less communication and fewer tokens and yielding a low false-positive rate; it is **complete**, applicable to both CFT and BFT systems; and it is **flexible**, featuring a modular, decoupled design that supports plug-and-play extension to other domains.

Moreover, our method should prioritize using computational resources to discover important **logic bugs**, rather than wasting tokens on **implementation bugs**.

3.2. Architecture

As shown in Figure 1, Agora is comprised of three core agents: (i) **Orchestrator** agent is responsible for workflow control and consists of two components. The bug-exploitation component leverages bugs previously discovered by Agora to more effectively trigger new bugs, while the state-analyzer component maintains the states of the other two agents and prevents Agora from repeatedly exploring identical bug scenarios; (ii) **Strategy** agent contains two components: a constraints-analyzer that analyzes the constraints of bug attack scenarios and supports both CFT and BFT protocols (**complete**), and an attack scenario-generator that produces protocol-specific scenarios to facilitate the discovery of deep logic bugs; (iii) **TestGen** agent employs a reflection-loop component to automatically generate, execute, and analyze unit tests, reducing false positives and synthesizing user-friendly bug reports based on the generated attack scenarios (**interpretability**).

Overall, this design, where one agent coordinates the workflow, one agent identifies protocol-level logic bug scenarios,

Table 1. Overall results for the baselines (top of the table) and Agora (bottom of the table). In total, the former correspond to implementation bugs ($\downarrow$, fewer is better, with darker blue indicating worse results), while the latter correspond to logic bugs ($\uparrow$, more is better, with darker red indicating better results). According to §3.1, fewer implementation bugs are preferred, whereas more logic bugs are desirable. The best results are highlighted in **bold**.

Bug Type	Implementation Bug ($\downarrow$)				Logic Bug ($\uparrow$)				Total
	Raft	EPaxos	HotStuff	BullShark	Raft	EPaxos	HotStuff	BullShark	
GPT-5.2 (OpenAI, 2025)	4	7	11	0	0	0	0	0	22/0
Gemini 3.0 Pro Preview (DeepMind, 2025)	0	0	0	0	0	0	0	0	0/0
Claude Sonnet 4.5 (Anthropic, 2025)	0	0	0	0	0	0	0	0	0/0
Qwen3 Coder 480B A35B (Bai et al., 2025)	0	3	0	0	0	0	0	0	3/0
Total (Baselines)	4	7	11	0	0	0	0	0	22/0
GPT-5.2 (OpenAI, 2025)	0	0	0	0	1	3	3	1	0/8
Gemini 3.0 Pro Preview (DeepMind, 2025)	0	0	0	0	1	6	3	1	0/11
Claude Sonnet 4.5 (Anthropic, 2025)	0	0	0	0	0	4	1	1	0/6
Qwen3 Coder 480B A35B (Bai et al., 2025)	0	0	0	0	1	6	2	0	0/9
Total (Agora)	0	0	0	0	1	9	4	1	0/15

and one agent focuses on test generation, effectively decouples system functionality, is composed of five components and enables **flexibility** and extensibility. §4 demonstrates the **efficiency** and **effectiveness** of Agora.

3.3. Knowledge Library

For bug patterns, we crawl GitHub repositories of consensus protocols to collect issues that have been confirmed as bugs and summarize them accordingly, filtering out low-level implementation bugs. In this way, we incorporate domain-specific knowledge of consensus protocols to help Agora understand protocol-level logic bugs and generate more effective attack scenarios. For constraints, we summarize the different settings and design choices of consensus protocols and, combined with the analysis of bug patterns, derive the distinct conditions under which bugs are triggered in CFT and BFT systems. For example, assuming malicious behavior of nodes in a CFT system can easily violate safety properties; however, such assumptions are impractical in real deployments and would unnecessarily waste computational resources.

By combining bug patterns and constraints, we introduce a succinct knowledge library for Agora that only provides the necessary domain knowledge for Agora. The details of bug patterns are presented in Appendix C and the details of constraints are in Appendix D.

3.4. Memory

In our system, the memory maintained by Agora consists only of repository knowledge and bug patterns. Bug patterns are described in detail in §3.3. Repository knowledge typically includes the repository name, the type of consensus protocol, protocol settings, implementation language, and testing methodologies. This information enables Agora

to efficiently support consensus protocols implemented in different programming languages and with diverse designs.

Both repository knowledge and bug patterns are shared across the agents of Agora for analysis and are lightweight and easy to maintain (the details are in Appendix E). Therefore, we store them in the system memory and maintain them within the execution environment of Agora LLM-based multi-agent system.

3.5. Workflow Details

Before reaching the termination condition or exhausting the budget, as shown in Figure 2, the workflow of Agora consists of 12 steps (subtasks).

In Step ①, Orchestrator performs its subtask O.1 and conducts bug exploitation using prompts. By analyzing previously discovered bugs, it provides better guidance for Strategy to derive new deep bugs based on prior bugs found from TestGen. In Step ②, Orchestrator performs subtask O.2 and uses prompts to analyze attack scenarios proposed earlier by Strategy together with bugs discovered by TestGen, producing a global state. In Step ③, Orchestrator performs subtask O.3 and sends the context information, i.e., the results of bug exploitation and state analysis, to Strategy.

In Step ④, Strategy performs subtask S.1 and analyzes protocol constraints using prompts. In Step ⑤, Strategy performs subtask S.2 and employs the context from bug exploitation and state analysis. In Step ⑥, Strategy performs subtask S.3 and, using prompts together with the results from Steps ④ and ⑤, generates attack scenarios. Specifically, for a given consensus protocol, it simulates scenarios by controlling node behaviors such as joining, going offline, crashing, or recovering, as well as manipulating message sending orders and conflict relationships to produce bug-prone scenarios. In Step ⑦, Strategy completes its subtask S.4 and sends

Table 2. Results for bug detection in ablation studies. We compare the original Agora with variants in which the corresponding components are removed.

Methods	Raft	EPaxos	HotStuff	BullShark	Total
Agora	1	6	3	1	11
w/o bug-exploitation	0	0	3	0	3
w/o state-analyzer	0	0	0	0	0
w/o constraints-analyzer	0	0	1	0	1
w/o scenario-generator	0	0	0	0	0
w/o reflection-loop	0	0	0	0	0

the generated attack scenarios to Orchestrator. In Step ⑧, Orchestrator completes its subtask O.4, updates the global state, and forwards the attack scenarios to TestGen.

In Step ⑨, TestGen performs its subtask T.1 and generates unit tests based on the received attack scenarios using prompts. In Step ⑩, TestGen performs subtask T.2 and executes the generated unit tests using prompts. In Step ⑪, TestGen performs subtask T.3 and analyzes the generated unit tests together with the execution results from Step ⑩ using prompts. If successful, this indicates that a logic bug in the consensus protocol has been discovered, and the workflow proceeds to Step ⑫. Otherwise, the process returns to Step ⑨. The number of iterations is bounded by a maximum value, `num_iter`; reaching this limit indicates that the current attack scenario is invalid and should be regenerated. In Step ⑫, TestGen completes its subtask T.4 and reports the successfully discovered bug to Orchestrator. It is worth noting that although Steps ⑨–⑪ are described as separate subtasks, they can be implemented using a single prompt that enables the TestGen agent to sequentially complete the three subtasks in the reflection loop.

We revisit HDT with Agora. In our framework, the attack scenarios correspond to the preconditions C , the unit tests generated by TestGen constitute the action sequence A , and the expected buggy behavior E and the oracle assertions O are contained in Steps ⑩ and ⑪ of TestGen. As a result, Agora effectively integrates domain knowledge of consensus protocols and the HDT paradigm into a multi-agent system. In summary, Orchestrator, Strategy, and TestGen are each composed of four subtasks. The prompt details of our agents are provided in Appendix I. The overall workflow is illustrated in Algorithm 1.

3.6. Communication Mechanism

Communication in Agora involves the memory, the global state, and the consensus protocol code repository.

As discussed in §3.4, the memory is lightweight and is accessed by the agents only once during the entire execution. By maintaining the global state within Orchestrator, Strategy does not need to track previously generated attack scenarios or discovered bugs and can easily retrieve

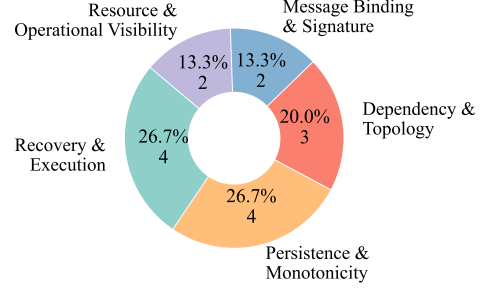


Figure 3. Distribution of discovered 15 protocol-level logic bugs of Agora by category.

the required state information from Orchestrator. Similarly, TestGen obtains the attack scenarios through Orchestrator. As a result, Strategy and TestGen do not need to manage contextual information and can focus solely on their respective tasks, while Orchestrator maintains the system-wide state and effectively manages long-horizon objectives. The global state itself is succinct, and its transmission occurs only between Orchestrator and Strategy or TestGen, without direct communication between Strategy and TestGen. Such communication is necessary to maintain system consistency and cannot be further reduced. In Agora, agents can access the consensus protocol code repositories via the Model Context Protocol (MCP) (Anthropic, 2024).

Overall, this design keeps the system concise, with communication limited to essential information, resulting in low communication overhead.

4. Experiments

4.1. Experiment Setup

Baselines. Existing studies that leverage LLMs for bug detection either lack integration with multi-agent systems or do not focus on deep logic-level bugs (Kaniewski et al., 2025; Sheng et al., 2025). To address these limitations and obtain appropriate comparison baselines, we adapt ReAct (Yao et al., 2023) by augmenting it with code-reading tools, and implement it using state-of-the-art code-oriented large language models as baselines. Specifically, we evaluate three commercial LLMs, Claude Sonnet 4.5 (Anthropic, 2025), Gemini 3.0 Pro Preview (DeepMind, 2025), and GPT-5.2 (OpenAI, 2025), as well as one open-source model, Qwen3 Coder 480B A35B (Bai et al., 2025).

Implementation Details. For our implementation, we access all models via APIs, eliminating the need for GPUs. For all large language models, we set the temperature to 0.2. In each experiment, both Agora and the baselines are allowed to run for three hours. For Agora, the number of reflection loop iterations (`num_iter`) is set to 50. The

implementation and artifact details are in Appendix A.

Code Repositories. We evaluate our approach on four popular protocol code repositories, covering two BFT and two CFT protocols, including Raft (etcd (etcd-io, 2026)), EPaxos (efficientEPaxos (efficient, 2026)), HotStuff (relab-HotStuff (relab, 2026)), and BullShark (sui (MystenLabs, 2026)). Among these, etcd and sui are production-level code repositories. Details of these repositories are in Appendix F.

4.2. Overall Results

Table 1 presents the overall experimental results of Agora compared with the baselines. From the results, we observe that regardless of the underlying LLM, the baselines are only able to identify simple implementation-level bugs, whereas our Agora, **by design**, consistently discovers protocol-level logic bugs and does not report implementation bugs.

Implementation-level bugs can typically be detected through static analysis and often result in issues such as integer overflows or memory violations. In contrast, logic bugs are significantly more subtle and may compromise system correctness, data consistency, or transaction security, leading to severe real-world consequences. Therefore, Agora explicitly focuses on deep protocol logic bugs rather than implementation-level issues. Detailed examples of the two bug types are provided in Appendix G. The experimental results demonstrate that our multi-agent design enables Agora to effectively focus on the detection of deep logic bugs, saving the computational resources and validating the effectiveness of our approach.

More specifically, we find that even when augmented with ReAct design, baselines using Gemini 3.0 Pro Preview and Claude Sonnet 4.5 fail to identify any implementation-level bugs. GPT-5.2 is the most effective baseline model, detecting 22 bugs across three repositories. Qwen3 Codr 480B A35B, possibly benefiting from its optimization for code understanding, is able to detect three bugs in the EPaxos repository, all of which are also identified by GPT-5.2. In total, the baselines uncover 22 zero-day implementation bugs, which we report as an additional contribution toward improving robustness of existing consensus implementations.

In contrast, Agora achieves the best performance when built upon Gemini 3.0 Pro Preview, identifying 11 protocol-level logic bugs across four widely used repositories, including production-level systems. This result is particularly notable given that the Gemini-based baseline fails to detect any bugs. Moreover, Agora consistently discovers logic bugs when instantiated with each of the other three LLMs, further demonstrating the robustness and generality of our approach. As some bugs are discovered by multiple LLM-backed instances, Agora identifies a total of 15 unique zero-day logic bugs, substantially enhancing the security of distributed con-

sensus systems. These results further validate the effectiveness of integrating our framework with the HDT paradigm.

Practical Usage. We further analyze all the 46 bug reports generated by the TestGen agent in Agora across the four LLMs. Among them, 34 reports correspond to real protocol-level logic bugs, resulting in a false positive rate of **only 26.1%**. This low false positive rate indicates that Agora does not require extensive manual filtering, making it a highly efficient and practical tool for engineers and researchers to discover deep logic bugs in real-world systems.

Bug Summary. Based on the categories of consensus logic bugs, we further analyze the 15 bugs discovered by Agora. As shown in Figure 3, violations are observed across all bug categories, demonstrating that Agora is capable of triggering a wide range of deep protocol-level bugs. Specifically, Recovery & Execution Divergence bugs account for four cases (26.7%), tying with Persistence & Monotonicity Violations as the most prevalent category. In addition, Message Binding & Signature Violations, Resource & Operational Visibility Violations, and Dependency & Topology Flaws account for 2, 2, and 3 bugs, respectively. Detailed examples of logic bugs from different categories are in Appendix H.

4.3. Ablation Studies

In this subsection, we analyze the impact of the five components of Agora on both bug discovery effectiveness and the computational resources required to discover bugs. Following the same experimental settings as before, we conduct ablation studies and select Gemini 3.0 Pro Preview as the underlying LLM, as it achieves the best overall performance under overall experiments in §4.2.

Summary. Table 2 summarizes the bug discovery results in the ablation study. The results show that only the complete Agora is able to discover the largest number of bugs (11 bugs), while removing any single component leads to degraded performance (at most 3 bugs). Overall, ablating any component reduces effectiveness by **73–100%**. Figure 4a shows the total token consumption in our ablation studies. Although removing the `bug-exploitation` and `reflection-loop` components can slightly reduce token usage (43.8M and 46.9M tokens in total, respectively, compared to 58.6M for the original configuration), this significantly hampers the discovery of useful bugs. Removing any of the other components results in higher token consumption, with the most significant increase observed when removing the `scenario-generator` (we will discuss later). Combined with Table 2, these results demonstrate that our modular design enables Agora to discover more meaningful bugs with fewer tokens, validating the effectiveness of our proposed components. Figure 4b further reports the number of tokens required to discover each bug under different ablation settings. Since majority of experiments

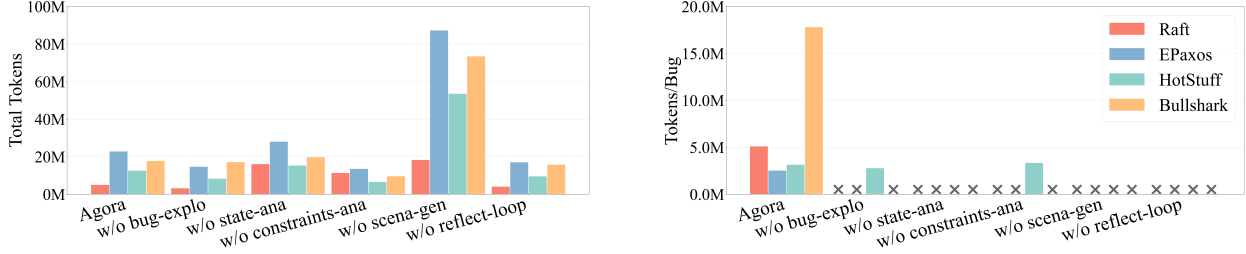


Figure 4. The left (a) shows the total token consumption in the ablation studies. The right (b) shows the average token consumption for each discovered bug in the ablation studies, where “cross” indicates that no bugs are discovered.

fail to discover any bugs, Agora is the most efficient, discovering one critical logic bug with an average of 5.32M tokens (approximately \$40). Identifying such protocol-level logic bugs typically requires years of effort from experienced engineers, making Agora highly cost-effective in practice.

Bug-exploitation. Removing bug-exploitation component is still able to discover a small number of bugs (3 bugs) in HotStuff, but fails to identify logic bugs in other repositories. Moreover, it cannot further explore deeper bugs based on previously discovered ones, resulting in significant token waste. Detailed examples of deeper bugs triggered through bug exploitation are provided in Appendix G.2.3. These results demonstrate that bug-exploitation enables Agora to more effectively discover deeper protocol-level bugs.

State-analyzer. Without state-analyzer component, Agora fails to discover any bugs. Further analysis shows that the system loses essential execution context, repeatedly retries similar scenarios, consumes excessive tokens (79.6M), and is unable to uncover logic bugs.

Constraints-analyzer. Removing constraints-analyzer component allows the system to discover a single bug in HotStuff (BFT protocol), but leads to a large number of false positives when testing CFT systems. In our experiments, this configuration generates **107** CFT bug reports, none of which correspond to positive logic bugs after manual inspection. This highlights the necessity of integrating the constraints-analyzer into Agora. Detailed differences between BFT and CFT bugs are discussed in Appendix G.2.1 and Appendix G.2.2.

Scenario-generator. When the scenario-generator component is removed, Agora regresses to baseline behavior. As shown in Figure 4a, this configuration incurs the highest token consumption (233.1M tokens, **3.98** times of original), discovers numerous implementation-level bugs, yet fails to identify any protocol-level logic bugs. In this setting, we uncover **29** zero-day implementation-level bugs. Although this demonstrates the effectiveness of the remaining components in enabling iterative bug discovery, it wastes tokens and fails to discover deep bugs targeted by our design goal.

Reflection-loop. Removing reflection-loop component prevents Agora from generating correct unit test files, thereby breaking the iterative discovery process for logic bugs. This results in wasted tokens, human power and reduced efficiency due to the generation of **173** problematic bug reports.

In summary, Orchestrator agent leverages the bug-exploitation and state-analyzer component to reduce context usage and communication overhead; Strategy agent generates effective attack scenarios for discovering deep bugs and adapts to different protocol types through the constraints-analyzer; and TestGen agent relies on the reflection-loop component to generate correct test cases. Together, these components ensure that Agora achieves the key properties of interpretability, efficiency, completeness, and flexibility.

5. Limitations and Discussions

Although our system has demonstrated significant effectiveness and has uncovered many zero-day logic bugs in consensus implementations, it still requires a certain amount of human knowledge. In the future, we aim to further improve its level of automation. Due to the inherent difficulty of identifying consensus bugs, we find 15 bug instances. In future work, we plan to extend our bug exploitation techniques and deploy our experiments on a broader range of consensus protocol codebases to collect sufficient data. With more data, we intend to apply Supervised Fine-Tuning (Wei et al., 2022; Ouyang et al., 2022) to enhance our system, and further generate trajectories to finetune our agent system using Direct Preference Optimization (Rafailov et al., 2023).

Moreover, we plan to generalize Agora, leveraging HDT and our decoupled design, to other debugging domains where both protocols and implementations are highly coupled and complex, such as concurrency control (Tan et al., 2020; Clark et al., 2024) and smart contracts (Sun et al., 2024), which are prone to causing severe economic and security issues in real-world systems.

6. Conclusion

In this paper, we combine specific domain knowledge with an LLM-based multi-agent system and, following the HDT paradigm, propose the Agora framework to detect consensus protocol-level logic bugs. Our approach is applicable to both research prototypes and production-level code repositories. By generating targeted attack scenarios, Agora is able to uncover deep, high-impact vulnerabilities that pose serious real-world threats. The system supports multiple programming languages, significantly enhancing the security of distributed systems, and demonstrates clear advantages over existing baselines. Benefiting from our cleanly decoupled design, our multi-agent framework not only satisfies key properties such as effectiveness, interpretability, efficiency, completeness, and flexibility, but also provides a strong foundation for future extension to other domain knowledge-intensive bug detection tasks.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There may be some potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Achiam, O. J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., and et.al., J. A. Gpt-4 technical report. 2023.
- Anthropic. Model context protocol (mcp) specification. <https://modelcontextprotocol.io>, 2024. Accessed: 2026-01-29.
- Anthropic. Claude sonnet 4.5 system card. Technical report, Anthropic, 2025. URL <https://www.anthropic.com/claude-sonnet-4-5-system-card>. Focuses on autonomous agents and tool-use capabilities.
- Bai, J., Chu, Y., Duan, Z., Fan, K., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>. Specifically covers the 480B MoE architecture and Coder variants.
- Cao, J., Ma, Y., Li, X., Ren, Q., and Chen, X. Task-specific efficiency analysis: When small language models outperform large language models, 2026. URL <https://arxiv.org/abs/2603.21389>.
- Castro, M. Practical byzantine fault tolerance. In *USENIX Symposium on Operating Systems Design and Implementation*, 1999.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Qian, C., Chan, C.-M., Qin, Y., Lu, Y.-T., Xie, R., Liu, Z., Sun, M., and Zhou, J. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. *International Conference on Learning Representations*, 2023. URL <https://api.semanticscholar.org/CorpusID:261048935>.
- Cheng, Y., Zhang, C., Zhang, Z., Meng, X., Hong, S., Li, W., Wang, Z., Wang, Z., Yin, F., Zhao, J., et al. Exploring large language model based intelligent agents: Definitions, methods, and prospects. *arXiv preprint arXiv:2401.03428*, 2024.
- Clark, J., Donaldson, A. F., Wickerson, J., and Rigger, M. Validating database system isolation level implementations with version certificate recovery. *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024. URL <https://api.semanticscholar.org/CorpusID:269262064>.
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., et al. Spanner: Google’s globally distributed database. *ACM Transactions on Computer Systems (TOCS)*, 31(3):1–22, 2013.
- Danezis, G., Kokoris-Kogias, E., Sonnino, A., and Spiegelman, A. Narwhal and tusk: a dag-based mempool and efficient bft consensus. *Proceedings of the Seventeenth European Conference on Computer Systems*, 2021.
- DeepMind, G. Gemini 3 technical report: Scaling native multimodal reasoning. Technical report, Google, 2025. URL https://ai.google.dev/static/gemini_3_report.pdf. Includes details on Deep Think reasoning modes.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J.-M., Zhang, R., Xu, R., Zhu, Q., Ma, S., and et.al., P. W. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *ArXiv*, abs/2501.12948, 2025.
- Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Aloomair, B., Wagner, D. A., Ray, B., and Chen, Y. Vulnerability detection with code language models: How far are we? *IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 1729–1741, 2025. URL <https://api.semanticscholar.org/CorpusID:268724273>.

- paxos consensus protocol.
- <https://github.com/efficient/epaxos>
- , 2026. GitHub repository (accessed 2026-01-09).
- etcd-io. raft: Raft library for maintaining a replicated state machine. <https://github.com/etcd-io/raft>, 2026. GitHub repository (accessed 2026-01-09).
- Feng, Z. Codebert: A pre-trained model for program-ming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.
- Fourney, A., Bansal, G., Mozannar, H., Tan, C., Salinas, E., Zhu, E. E., Niedtner, F., Proebsting, G., Bassman, G., Gerrits, J., Alber, J., Chang, P., Loynd, R., West, R., Dibia, V., Awadallah, A. M., Kamar, E., Hosn, R., and Amershi, S. Magentic-one: A generalist multi-agent system for solving complex tasks. *ArXiv*, abs/2411.04468, 2024. URL <https://api.semanticscholar.org/CorpusID:273877854>.
- Giridharan, N., Kokoris-Kogias, L., Sonnino, A., and Spiegelman, A. Bullshark: Dag bft protocols made practical. *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022.
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N., Wiest, O., and Zhang, X. Large language model based multi-agents: A survey of progress and challenges. In *International Joint Conference on Artificial Intelligence*, 2024. URL <https://api.semanticscholar.org/CorpusID:267412980>.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*, 2023.
- Hu, M., Zhou, Y., Fan, W., Nie, Y., Xia, B., Sun, T., Ye, Z., Jin, Z., Li, Y., Chen, Q., Zhang, Z., Wang, Y., Ye, Q., Ghanem, B., Luo, P., and Li, G. Owl: Optimized workforce learning for general multi-agent assistance in real-world task automation. *Neural Information Processing Systems*, 2025. URL <https://api.semanticscholar.org/CorpusID:279071173>.
- Hunt, P., Konar, M., Junqueira, F. P., and Reed, B. Zookeeper: Wait-free coordination for internet-scale systems. In *2010 USENIX annual technical conference*, pp. 11–11, 2010.
- Kaniewski, S., Schmidt, F., Enzweiler, M., Menth, M., and Heer, T. A systematic literature review on detecting software vulnerabilities with large language models. *ArXiv*, abs/2507.22659, 2025. URL <https://api.semanticscholar.org/CorpusID:280391565>.
- Keidar, I., Kokoris-Kogias, E., Naor, O., and Spiegelman, A. All you need is dag. *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, 2021.
- Kell, D. B. and Oliver, S. G. Here is the evidence, now what is the hypothesis? the complementary roles of inductive and hypothesis-driven science in the post-genomic era. *BioEssays : news and reviews in molecular, cellular and developmental biology*, 26 1:99–105, 2004.
- Lamport, L. The part-time parliament. *ACM Transactions on Computer Systems (TOCS)*, 1998.
- Lamport, L. Paxos made simple. 2001.
- Lamport, L. Fast paxos. *Distributed Computing*, 2006.
- Lamport, L. Byzantizing paxos by refinement. In *International Symposium on Distributed Computing*, 2011.
- Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. Camel: Communicative agents for ”mind” exploration of large language model society. In *Neural Information Processing Systems*, 2023.
- Moraru, I., Andersen, D. G., and Kaminsky, M. There is more consensus in egalitarian parliaments. *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, 2013.
- MystenLabs. Sui: Next-generation smart contract platform with high throughput and low latency. <https://github.com/MystenLabs/sui>, 2026. GitHub repository (accessed 2026-01-09).
- Nakamoto, S. Bitcoin: A peer-to-peer electronic cash system, 2008.
- Ongaro, D. and Ousterhout, J. K. In search of an understandable consensus algorithm. In *USENIX Annual Technical Conference*, 2014.
- OpenAI. Gpt-5.2 system card: Evaluating expert-level reasoning and persistence. Technical report, OpenAI, 2025. URL <https://openai.com/index/gpt-5-2-system-card>. Details on the reasoning model and long-horizon task performance.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L. E., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. J. Training language models to follow instructions with human feedback. *Neural Information Processing Systems*,

2022. URL <https://api.semanticscholar.org/CorpusID:246426909>.
- Pearce, H., Tan, B., and Dolan-Gavitt, B. Asleep at the keyboard? assessing the security of github copilot’s code contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (S&P)*, 2022.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., and Sun, M. Chatdev: Communicative agents for software development. In *Annual Meeting of the Association for Computational Linguistics*, 2023. URL <https://api.semanticscholar.org/CorpusID:270257715>.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Neural Information Processing Systems*, 2023. URL <https://api.semanticscholar.org/CorpusID:258959321>.
- relab. hotstuff: A framework for experimenting with the hotstuff consensus protocol. <https://github.com/relab/hotstuff>, 2026. GitHub repository (accessed 2026-01-09).
- Sheng, Z., Chen, Z., Gu, S., Huang, H., Gu, G., and Huang, J. Llm in software security: A survey of vulnerability detection techniques and insights. *ACM Computing Surveys*, 58:1 – 35, 2025. URL <https://api.semanticscholar.org/CorpusID:276258857>.
- Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., and Liu, Y. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. *IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pp. 2048–2060, 2024. URL <https://api.semanticscholar.org/CorpusID:260682978>.
- Taft, R., Sharif, I., Matei, A., VanBenschoten, N., Lewis, J., Grieger, T., Niemi, K., Woods, A., Birzin, A., Poss, R., et al. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, pp. 1493–1509, 2020.
- Tan, C., Zhao, C., Mu, S., and Walfish, M. Cobra: Making transactional key-value stores verifiably serializable. In *USENIX Symposium on Operating Systems Design and Implementation*, 2020. URL <https://api.semanticscholar.org/CorpusID:229250750>.
- Wang, C., Wu, Q., and the AG2 Community. AG2: Open-source agents for ai agents. <https://github.com/ag2ai/ag2>, 2024a. Documentation available at <https://docs.ag2.ai/>.
- Wang, C., Zhang, W., Su, Z., Xu, X., Xie, X., and Zhang, X. Llmdfa: Analyzing dataflow in code with large language models. *Advances in Neural Information Processing Systems*, 2024b. URL <https://api.semanticscholar.org/CorpusID:267740254>.
- Wang, C., Zhang, W., Su, Z., Xu, X., and Zhang, X. Sanitizing large language models in bug detection with data-flow. In *Conference on Empirical Methods in Natural Language Processing*, 2024c. URL <https://api.semanticscholar.org/CorpusID:274023938>.
- Wei, J., Bosma, M., Zhao, V., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=gEZrGCozdqR>.
- Wen, X., Gao, C., Gao, S., Xiao, Y., and Lyu, M. R. Scale: Constructing structured natural language comment trees for software vulnerability detection. *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024. URL <https://api.semanticscholar.org/CorpusID:268733181>.
- Yang, C., Zhao, Z., Xie, Z., Li, H., and Zhang, L. Knighter: Transforming static analysis with llm-synthesized checkers. *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025a.
- Yang, C., Zhao, Z., and Zhang, L. Kernelgpt: Enhanced kernel fuzzing via large language models. *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025b.
- Yang, Y., Chai, H., Shao, S., Song, Y., Qi, S., Rui, R., and Zhang, W. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. *Advances in Neural Information Processing Systems*, 2025c. URL <https://api.semanticscholar.org/CorpusID:277468263>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2023.

- Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., and Abraham, I. Hotstuff: Bft consensus in the lens of blockchain. *arXiv: Distributed, Parallel, and Cluster Computing*, 2018.
- Zeng, Y., Yu, W., and et.al. Bridging the editing gap in llms: Fineedit for precise and targeted text modifications. *EMNLP*, 2025.
- Zhang, C., Yang, K., Hu, S., Wang, Z., Li, G., Sun, Y., Zhang, C., Zhang, Z., Liu, A., Zhu, S.-C., Chang, X., Zhang, J., Yin, F., Liang, Y., and Yang, Y. Proagent: Building proactive cooperative agents with large language models. In *AAAI Conference on Artificial Intelligence*, 2023.
- Zheng, J., Lin, Q., Xu, J., Wei, C., Zeng, C., Yang, P., and Zhang, Y. Paxosstore: high-availability storage made practical in wechat. *Proceedings of the VLDB Endowment*, 10(12):1730–1741, 2017.
- Zhou, J. and et.al., M. X. Foundationdb: A distributed unbundled transactional key value store. *SIGMOD*, 2021.

Appendix

Contents

- Section A: Artifact Details
- Section B: Mathematical Modeling of the Multi-Agent System
- Section C: Bug Pattern Details
- Section D: Constraints Details
- Section E: Memory Details
- Section F: Codebase Details
- Section G: Two Types of Bug Example
- Section H: Bug Example for Five Classes
- Section I: Prompt Example

A. Artifact Details

Our codebase is available at <https://github.com/lebronlambert/Agora>. Similar to Agora, our baselines also incorporate essential information, such as repository knowledge to get a fair comparison. Setting up the environment is straightforward with the provided README file. In the README, you will find all the necessary guidelines to use the bash scripts to reproduce the main results from our experiments.

B. Mathematical Modeling of the Multi-Agent System

LLMs power a wide variety of applications (Cao et al., 2026; Zeng et al., 2025). Specifically, in a formal sense, we characterize an autonomous LLM-powered entity as a quintuple $\mathcal{E} = \langle \mathcal{B}, \mathcal{P}, \mathcal{A}, \Omega, \Gamma \rangle$. Here, $\mathcal{B}$ represents the core linguistic brain (the LLM), $\mathcal{P}$ signifies the perceptual manifold of the target environment, and $\mathcal{A}$ denotes the repertoire of executable operations (tools). The mapping function $\Omega : \mathcal{P} \rightarrow \mathcal{A}$ acts as the decision logic, while Γ encapsulates the persistent memory state. The operational trajectory of such an entity follows a recursive observe-deliberate-act cycle: at any discrete temporal index k , the entity ingests a sensory input $p_k \in \mathcal{P}$, leverages $\mathcal{B}$ to synthesize a strategic intent based on Ω , and dispatches an intervention $a_k \in \mathcal{A}$ to modulate the system state. To tackle the intricate state spaces of consensus implementations, we extend this individual paradigm into a collaborative collective $\mathcal{G} = \{\mathcal{E}_1, \mathcal{E}_2, \dots, \mathcal{E}_m, \mathcal{X}\}$. In this formulation, each $\mathcal{E}_i$ represents a specialized functional unit, and $\mathcal{X}$ defines the coordination medium that facilitates information exchange and role synchronization. This multi-agent architecture transcends the cognitive boundaries of isolated models by decomposing complex protocol verification into modular, inter-dependent reasoning sub-tasks, a methodology that has proven more robust than monolithic approaches in recent literature, e.g., CAMEL (Li et al., 2023), MetaGPT (Hong et al., 2023), OWL (Hu et al., 2025).

C. Bug Pattern Details

We implemented a crawler to collect all **816** GitHub issues from the four code repositories. We then filtered the issues and identified **34** issues related to consensus protocol logic bugs, which were used to construct our bug patterns to help us integrate expert knowledge. Examples are shown below:

Bug Pattern Details Show Case 1

Pattern ID: Byzantine DoS via insufficient QC validation - single malicious node can stall consensus by forging high-view QCs

Protocol Type: BFT (HotStuff Variant)

Category: Security / Liveness / Insufficient Validation

Description: A critical vulnerability exists in the `AggregateQC` creation and verification process. A single Byzantine node can indefinitely stall consensus by providing a forged Quorum Certificate (QC) with a deceptively high view number.

Root Cause Analysis:

- **CreateAggregateQC** blindly trusts and collects QCs from `TimeoutMsg` payloads without individual verification.
- **VerifyAggregateQC** only validates the `highQC` (the QC with the highest view), creating a single point of failure.

Attack Scenario:

1. **Forgery:** A Byzantine node crafts a `TimeoutMsg` containing a forged QC with a very high view number.
2. **Aggregation:** The Leader trustingly includes this invalid `highQC` into the `AggregateQC`.
3. **Proposal:** The Leader proposes the new block/view containing this `AggregateQC`.
4. **Rejection:** Honest nodes attempt to verify the `AggregateQC`. Since `VerifyQuorumCert` (`highQC`) fails, the entire proposal is rejected.
5. **Stall:** As the leader cannot form a valid proposal that honest nodes accept, the consensus process halts.

Impact: Complete blockchain liveness failure. A single malicious actor can perpetually halt the network's progress.

Fix & Mitigation:

- **Individual Verification:** Modify `CreateAggregateQC` to independently validate every QC using `VerifyQuorumCert` before its inclusion.
- **Zero-Trust Policy:** Never trust aggregate structures from other nodes without verifying each component.

Context:

- **Reference:** [relab/hotstuff Issue #224](#)
- **Vulnerable Functions:** `CreateAggregateQC`, `VerifyAggregateQC`
- **Confirmed by:** meling (maintainer)

Bug Pattern Details Show Case 2

Pattern ID: `VerifyLeader` OOM - notify list grows unbounded when futures not removed from `repl.notify` on error

Protocol Type: CFT (Hashicorp Raft)

Bug Category: Liveness / Memory Leak / OOM

Description: In the `VerifyLeader` mechanism, a node failure triggers an unbounded memory growth. The `verifyLeader()` function registers a verify future `v` into the `repl.notify` list for each replica. When a heartbeat or `AppendEntries` returns an error, the cleanup process is incomplete, leading to a memory leak that eventually causes an Out-Of-Memory (OOM) crash.

Root Cause Analysis:

- **Asymmetric Cleanup:** The future `v` is tracked in two distinct structures: `leaderState.notify` and `repl.notify`.
- **Incomplete Removal:** On error, the future is only deleted from `leaderState.notify` but remains in `repl.notify`.
- **Resource Exhaustion:** The replication goroutine persists in holding references to failed or completed futures,

preventing Garbage Collection (GC) and causing the list to grow infinitely.

Lessons & Best Practices:

- **Symmetric Lifecycle:** Ensure that any future or callback registered in multiple structures is removed from *all* of them upon completion or failure.
- **Failure Path Testing:** Monitor memory usage specifically under failure conditions, as leaks often remain hidden during healthy cluster operations.
- **Cleanup Patterns:** Utilize `defer` or RAII-like patterns to guarantee cleanup across all possible execution paths.
- **Subscription Management:** For async notifications, consider explicit lifecycle management or weak references to manage subscription longevity.

Context:

- **Reference:** [Hashicorp Raft Issue #291](#)
- **Tags:** `cft`, `memory_leak`, `verify_leader`, `async_notify`

D. Constraints Details

Here, we provide examples of constraints generated for BFT and CFT systems, which help generate scenarios tailored to different types of consensus protocols.

Protocol Constraints & BFT Attack Model

Protocol Type Constraint: BFT (Byzantine Fault Tolerant)

This model assumes an environment capable of tolerating **Byzantine faults**, where nodes may behave maliciously or arbitrarily.

I. CFT-type Attack Vectors (Crash Faults):

- **Availability:** Node crash, recovery, and frequent restarts.
- **Network:** Partitions, significant latency, packet loss, and message reordering.
- **Logic:** Boundary condition exploits and duplicate message utilization.

II. BFT-specific Attack Vectors (Malicious Behavior):

- **Equivocation:** Sending conflicting votes/proposals for the same round/height to different peers.
- **Payload Tampering:** Injecting invalid, malformed, or tampered protocol messages.
- **Selective Broadcast:** Distributing messages to a subset of nodes to induce state inconsistency.
- **Strategic Withholding:** A malicious leader intentionally delays or withholds proposals to stall liveness.
- **State Forgery:** Crafting fraudulent certificates, votes, or cryptographic signatures.
- **Timing Attacks:** Exploiting vulnerabilities within the protocol's timing and timeout assumptions.

III. Attack Mindset & Objectives: Assuming control of up to f malicious nodes ($f < n/3$), the attacker aims to:

1. **Safety Violation:** Can honest nodes be coerced into accepting an invalid state?
2. **Liveness Violation:** Can the system be prevented from reaching consensus indefinitely?
3. **Consistency Violation:** Can different honest nodes be led to perceive divergent “truths”?

Note: Effective testing must simulate a clever attacker who strategically exploits the protocol’s specific logic weaknesses rather than just random fault injection.

Protocol Constraints & CFT Fault Model

Protocol Type Constraint: CFT (Crash Fault Tolerant)

This model assumes an environment that can only tolerate **crash faults**. All nodes are assumed to be honest; they either follow the protocol correctly or stop functioning entirely.

I. Allowed Attack Methods (Non-Malicious Faults):

- **Node Dynamics:** Random node crashes (immediate stop) and restarts (recovery with potential loss of in-memory state).
- **Network Adversity:** Network partitions, extreme communication delays, message loss, and packet reordering.
- **Implementation Limits:** Boundary conditions (overflows, null values, extreme inputs) and duplicate resource utilization.

II. Prohibited Attack Methods (Byzantine Behavior):

- **No Malicious Messages:** Sending forged, tampered, or intentionally malformed payloads is *strictly prohibited*.
- **No Equivocation:** Double voting or sending conflicting proposals in the same round is not allowed.
- **No Selective Broadcast:** Intentional inconsistency through partial message distribution is excluded.
- **No Identity Theft:** Actions involving signature forgery or impersonating other nodes are disallowed.

III. Core Assumptions: *In a CFT environment, the testing focus shifts from “malice” to “reliability”:*

1. **Binary State:** Nodes are either “Correct” (following the protocol) or “Failed” (stopped). There is no “half-dead” or Byzantine behavior.
2. **Honest Participation:** All participating nodes are assumed to act in good faith to reach a common consensus.
3. **Recovery Consistency:** The system must maintain safety and eventually regain liveness after network partitions heal or nodes recover.

Note: Testing for CFT protocols should exhaustively simulate unstable infrastructure and edge-case logic rather than adversarial intent.

E. Memory Details

The memory contains repository knowledge and bug patterns, with a small total size of about 360 KB. Examples of repository knowledge are shown below.

Table 3. Codebase information.

Repo	Type	Protocol	Lang	Stars	Forks
etcd (etcd-io, 2026)	CFT	Raft	Go	982	222
efficientEPaxos (efficient, 2026)	CFT	EPaxos	Go	631	138
relabHotStuff (relab, 2026)	BFT	HotStuff	Go	217	67
sui (MystenLabs, 2026)	BFT	BullShark	Rust	7.6k	11.7k

Repository Knowledge: etcd Raft (Go)

Core Testing Helpers & API:

- `newNetwork(peers...)`: Creates an in-memory Raft network simulation environment, supporting fault injection and message dropping.
- `newTestRaft(id, election, heartbeat, storage)`: Initializes a Raft instance for white-box testing with specific timing parameters.
- `nextEnts(r, s)`: Returns entries that are committed but not yet applied, and advances the applied index.
- **MemoryStorage**: In-memory implementation of the `Storage` interface. Key operations like `Append`, `CreateSnapshot`, and `Compact` must respect boundary conditions (e.g., `Compact` cannot exceed `lastIndex`).

Key States & Message Types:

- **States**: `StateFollower` (0), `StateCandidate` (1), `StateLeader` (2), `StatePreCandidate` (3).
- **Local Triggers**: `MsgHup` (triggers election), `MsgBeat` (triggers heartbeat broadcast).
- **Replication**: `MsgProp` (proposal), `MsgApp` (append entries), `MsgAppResp` (append response).
- **Control**: `MsgHeartbeat`, `MsgSnap` (snapshot transfer), `MsgVote` (solicit vote).

Lessons Learned & Common Pitfalls:

- **Implicit No-op Entry**: Upon becoming a Leader (`becomeLeader`), the node automatically appends an empty **no-op** entry. This must be accounted for when calculating expected `lastIndex`.
- **Single-Node Quorum**: In a 1-node cluster, entries are committed immediately upon append, but still require a call to `nextEnts` to drive the application logic.
- **Flow Control (Inflight)**: When `MaxInflight=1`, the system is highly sensitive. Probe responses (empty `MsgApp`) must be handled correctly to unpause the pipeline and prevent `PipelineStall`.
- **Snapshot Boundaries**: `MemoryStorage` maintains a dummy entry at `FirstIndex()-1`. If `HardState.Commit` is less than `FirstIndex`, Raft will panic during initialization.
- **Learner Promotion Safety**: Promoting a Learner to a Voter while it has unacknowledged inflight messages can lead to state machine stalls (e.g., stuck in `StateProbe`) if the transition logic does not explicitly clear buffers.

Execution Context: Always run tests using the package context (`go test -v -run TestName .`) rather than standalone files (`go test file.go`) to ensure that unexported helper functions in `raft_test.go` are correctly linked.

F. Codebase Details

We evaluate our approach on four popular consensus protocol codebases, including both research prototypes and production-level repositories, as summarized in Table 3.

G. Two Types of Bug Example

G.1. Implementation Bug

Implementation Bug

Analysis: Incorrect RPC Message Dispatch (Copy-Paste Error)

Problem Description A critical logic error exists in the `bcastTryPreAccept` function. Despite correctly populating a `TryPreAccept` structure (`tpa`), the code erroneously references an unrelated `PreAccept` structure (`pa`) during dispatch.

Affected Components

- **File:** `src/epaxos/epaxos.go`
- **Function:** `bcastTryPreAccept` (Lines 580–595)

Root Cause Analysis In `epaxos.go`, the function spends lines 588–594 filling the fields of `tpa`. However, at line 595, the code executes: `args := &pa`. Since `pa` represents a different RPC schema, the broadcast sends stale or misaligned protocol data.

Attack Scenario / Impact

1. **Mangled Communication:** The leader broadcasts a message intended as `TryPreAccept`, but the payload is cast from a `PreAccept` structure.
2. **Protocol Collapse:**
 - **Recovery Failure:** Doubtful instances remain unresolved, leading to a permanent Liveness hang.
 - **State Machine Divergence:** Peers may accept inconsistent dependency graphs based on outdated `pa` data.

Corrective Action Update the reference at line 595 to the correct initialized structure:

Change `&pa` → `&tpa`

G.2. Logic Bug

G.2.1. BFT BUG

BFT Bug

Analysis: Case Study: Safety Violation via Cascading Forks in HotStuff

Affected Components

- **Files:** `protocol/rules/chainedhotstuff.go`, `protocol/consensus/voter.go`
- **Modules:** `ChainedHotStuff.VoteRule()`, `CommitRule()`

Root Cause Analysis The vulnerability stems from an interplay between weak liveness checks and the 3-chain commitment logic:

- **Vulnerable Vote Rule:** The liveness condition (`QC.View > bLock.View`) is too permissive. If a node's

bLock is lagged (e.g., after a reboot), any Quorum Certificate (QC) from a higher view can bypass the safety check, even if it extends an ancient branch.

- **Context-Free Commit Rule:** The `CommitRule` strictly validates the 3-chain structure (consecutive views and matching parent hashes) but fails to ensure the chain is part of the "canonical" branch.
- **Mutable bLock:** bLock updates whenever a 2-chain is formed on *any* fork, lowering the barrier for Byzantine leaders to push a malicious branch.

Attack Scenario (n=4, f=1)

1. **State:** Honest nodes hold a high QC for B_3 (Chain: $G \rightarrow B_1 \rightarrow B_2 \rightarrow B_3$). A target honest node has `bLock = Genesis`.
2. **Step 1 (View 4):** A Byzantine leader proposes B'_4 extending B_1 . Since $QC_{B_1}.View(1) > bLock.View(0)$, the node votes, forming QC'_4 .
3. **Step 2 (Views 5–6):** Consecutive Byzantine leaders extend the fork to B'_5 and B'_6 . The node's `bLock` is updated to B'_4 , facilitating the fork's progression.
4. **Step 3 (View 7):** Upon receiving B'_7 , the `CommitRule` detects a valid 3-chain ($B'_4 \rightarrow B'_5 \rightarrow B'_6$). Consequently, B'_4 is committed, bypassing the correct $B_2 \rightarrow B_3$ sequence.

Impact & Security Analysis This attack is critical as it leads to a **Safety Violation**. Unlike transient liveness issues, this results in permanent state pollution and ledger divergence across honest nodes. The attack is feasible in scenarios involving node restarts or state synchronization lags, especially when the protocol allows a Byzantine node to be elected as a leader for consecutive views.

Verification The exploit was successfully reproduced using:

```
go test -v -run TestByzantineLeaderBuildsCascadingForkAcrossViews
```

G.2.2. CFT BUG

CFT Bug

Analysis: Case Study: Dependency Divergence during EPaxos Recovery

Affected Components

- **File:** `src/epaxos/epaxos.go`
- **Modules:** `handlePrepareReply()`, `startPhase1()`, and `updateAttributes()`

Root Cause Analysis The vulnerability is rooted in the improper re-computation of command attributes during the recovery phase. When a replica attempts to recover an instance whose Fast Path status is ambiguous, it incorrectly invokes `startPhase1`, which triggers a fresh call to `updateAttributes`.

- **Attribute Volatility:** Instead of preserving the original dependencies recorded during the pre-accept phase, `startPhase1` re-scans the local log for conflicting commands.
- **Dependency Smuggling:** If a new, conflicting command has arrived during the leader's downtime, it will be "smuggled" into the recovered command's dependency set. This creates a discrepancy between the dependencies used during the original Fast Path execution and the final committed state.

Attack Scenario (3 Nodes: R_0, R_1, R_2)

1. **Fast Path Execution:** R_0 proposes Cmd_A (Key="x"). R_0 and R_1 accept it with `deps = {}`. R_0 reaches a Fast Quorum, commits, and executes Cmd_A locally ($x = 1$).

2. **Leader Crash:** R_0 crashes immediately before broadcasting the `Commit` message to others.
3. **Conflict Injection:** R_2 proposes Cmd_B (conflicting with Cmd_A). R_2 and R_1 accept Cmd_B . Crucially, R_1 now sees Cmd_B in its log.
4. **Vulnerability Trigger:** R_1 initiates recovery for Cmd_A . Failing to observe a Fast Quorum, it executes `startPhase1`. The `updateAttributes` function sees Cmd_B and re-calculates Cmd_A 's dependencies as $deps = \{Cmd_B\}$.
5. **State Divergence:** The cluster eventually commits Cmd_A with $deps = \{Cmd_B\}$, implying an execution order of $B \rightarrow A$. However, the failed leader R_0 has already executed A with $deps = \{\}$.

Impact & Security Analysis This bug represents a **Linearizability Violation**. Upon recovery, R_0 will find its local execution history contradicts the globally committed serialized order. In high-throughput systems with frequent leader transitions or network partitions, this divergence leads to permanent database corruption and "split-brain" states at the application level.

Verification The divergence can be reproduced by a controlled test case where:

- A specific node is isolated after a Fast Path commit.
- A conflicting command is injected into the remaining quorum.
- Recovery is forced on the surviving nodes.

G.2.3. BUG TRIGGERS FROM BUG EXPLOITATION

Bug from Bug Exploitation

Analysis: Case Study: History Erasure via Deep Fork from Genesis

Affected Components

- **Files:** `protocol/rules/chainedhotstuff.go`, `protocol/consensus/voter.go`
- **Modules:** `ChainedHotStuff.VoteRule()`, `NewChainedHotStuff()`

Root Cause Analysis This vulnerability represents the most extreme manifestation of "HighQC Ignorance." It exploits the discrepancy between the `HighQC` (the highest known certificate) and `bLock` (the locked block) during edge cases such as node recovery.

- **Lock Amnesia:** The current implementation stores `bLock` in memory without persistence. Upon a node restart or crash-recovery, `bLock` is reset to `Genesis` (`View 0`).
- **Trivial Safety Satisfaction:** The Safety condition in `VoteRule` checks if a proposal extends the `bLock`. When `bLock = Genesis`, the `blockchain.Extends(block, Genesis)` call returns `true` for any valid block in the system, effectively neutralizing the safety guardrail.
- **Liveness Bypass:** While the Liveness condition (`qcBlock.View > bLock.View`) typically prevents old forks, a leader can satisfy it using any QC with `View > 0` (e.g., QC_{B1}) while simultaneously targeting nodes that have just reset their locks.

Attack Scenario: Full History Erasure Assume an established chain: $G \rightarrow B_1 \rightarrow B_2 \rightarrow B_3 \rightarrow B_4 \rightarrow B_5$ with $\text{HighQC} = QC_{B5}$.

1. **Target Condition:** A subset of honest nodes restart, causing their `bLock` to revert to `Genesis`.
2. **The Deep Fork:** A Byzantine leader ignores the entire history and proposes B'_6 extending directly from `Genesis` (Parent: `Genesis`, QC: QC_G).

3. Validation Failure:

- **Liveness:** If using QC_{B_1} , $QC_{B_1}.View(1) > bLock.View(0)$, condition passes.
- **Safety:** Extends (B'_6 , Genesis) is true, condition passes.

4. **Outcome:** Honest nodes vote for B'_6 , enabling the Byzantine leader to build an alternative chain ($G \rightarrow B'_6 \rightarrow B'_7 \dots$) that competes with the legitimate history, potentially leading to a massive state roll-back if the fork gains quorum.

Impact & Security Analysis This bug exposes the system to **Total History Erasure**. Unlike standard forks that deviate by a few blocks, this "Deep Fork" allows an attacker to rewrite the ledger from the Genesis block. While HotStuff's voting rules (one vote per view) prevent simultaneous double-commits within the same view, the "Lock Amnesia" creates a window where a Byzantine leader can force honest nodes to abandon the canonical chain in favor of a malicious alternative, causing severe consistency issues and service disruption.

Verification Reproduced via:

```
go test -v -run TestByzantineLeaderBuildsDeepForkFromGenesis
```

Result: BUG CONFIRMED: Voter accepted proposal extending Genesis despite HighQC being at View 5.

This bug is uncovered through further exploration of the bug described below.

Bug: Design Vulnerability in Consensus Voting Logic

Analysis: Case Study: Byzantine Leader Proposes Block Ignoring HighQC

Affected Components

- **Files:** `protocol/rules/chainedhotstuff.go`, `protocol/consensus/voter.go`
- **Modules:** `ChainedHotStuff.VoteRule()`, `NewChainedHotStuff()`

Root Cause Analysis This issue stems from a design-implementation gap where `VoteRule` validates proposals against `bLock` (Locked Block) rather than the `HighQC` (Highest Quorum Certificate). Under specific edge cases—such as node recovery or state synchronization—`bLock` can become desynchronized from the actual chain height.

- **Implementation Logic:** Per the HotStuff paper, a vote is cast if:
 1. **Liveness:** `QC.View > bLock.View`
 2. **Safety:** The block extends `bLock`.
- **Persistence Failure:** `bLock` is initialized to `Genesis` and kept only in memory. It is updated during the `CommitRule` but is not persisted to disk.
- **The Vulnerability:** Upon restart, a node's `bLock` reverts to `Genesis` (View 0). Even if the blockchain is recovered, the node will accept any proposal that satisfies the Liveness condition against a view of 0, even if the proposal ignores the actual canonical tip (`HighQC`).

Attack Scenarios Scenario: Node Recovery / New Joiner

1. **State:** Chain exists as $G \rightarrow B_1 \rightarrow B_2 \rightarrow B_3$ ($HighQC = B_3$).
2. **Trigger:** A node restarts; `bLock` resets to `Genesis`.
3. **The Attack:** A Byzantine leader proposes B'_4 extending B_1 using QC_{B_1} .
4. **Validation:** `VoteRule` sees $QC_{B_1}.View(1) > bLock.View(0)$. The Liveness condition passes, and the honest node mistakenly votes for a fork that ignores B_3 .

Verification Reproduced via: `go test -v -run TestByzantineLeaderIgnoresHighQC ./protocol/consensus/`

```
BUG DETECTED: Voter.Verify() accepted malicious proposal extending
B1 instead of HighQC B3
BUG CONFIRMED: Honest node voted for malicious proposal!
Result: This allows a Byzantine leader to fork the chain and split
the quorum.
```

Impact & Security Analysis While this vulnerability strictly follows the HotStuff paper’s safety rules (preventing double-commits within the same view via `lastVotedView` checks), it creates significant **Liveness and Stability risks**:

- **Chain Forking:** Malicious leaders can force honest nodes to vote on stale branches.
- **Quorum Fragmentation:** Voting power is split between the canonical chain and the fork, stalling finality.
- **Increased Latency:** Recovery from such "HighQC ignorance" requires extra views to re-synchronize the locked state.

H. Bug Example for Five Classes

Recovery&Execution Divergence

Analysis: `CommittedCommandVanishing` (Loss of Committed State)

Affected Components

- **File:** `src/epaxos/epaxos.go`
- **Line Numbers:** 1542–1556
- **Logic:** The `else` branch in the recovery handler where the leader receives no information from the quorum.

Root Cause Analysis The vulnerability arises because the recovery leader assumes an instance is "empty" or "unused" if it receives no records from its recovery quorum. When this happens, it erroneously proposes a `NO-OP` to finalize the instance. This logic fails to account for **Amnesia** (node restarts losing volatile state) or **Network Partitions** that isolate the only nodes holding the original `Commit` record.

Attack Scenario (5 Nodes: R_0, R_1, R_2, R_3, R_4)

1. **Fast Path Execution:** R_0 proposes Cmd_A (`Key="k", Val=999`). Nodes R_0, R_1, R_2 accept it (Fast Quorum).
2. **Partial Commit:** R_0 sends a `Commit` message **only** to R_2 . R_2 marks the command as `Committed`.
3. **Compound Failure:**
 - R_0 crashes permanently.
 - R_1 crashes and reboots (Amnesia), wiping its log.
 - R_2 is partitioned from the rest of the cluster.
4. **Flawed Recovery:** R_3 initiates recovery for the same instance and queries $\{R_1, R_3, R_4\}$.
5. **Voiding the Instance:** R_1, R_3 , and R_4 all report no record. R_3 incorrectly concludes the instance was never used and proposes `NO-OP`.
6. **State Divergence:** R_1, R_3, R_4 commit and execute `NO-OP`. When the partition heals, R_2 has already executed Cmd_A , causing a permanent state divergence.

Impact & Risk Assessment

- **Outcome:** Violation of the "Stability" property and Linearizability.
- **Feasibility:** High in environments without strict persistence (Sync-to-Disk) and frequent network partitions.
- **Risk Level: Medium** (Requires specific failure interleaving).

Corrective Action The protocol should either:

- Strictly prohibit Amnesia by requiring persistent WAL (Write-Ahead Logs).
- Enhance the recovery logic to require an explicit "Proof of Non-Existence" before proposing a NO-OP.

Persistence & Monotonicity Violations**Analysis: Case Study: ExecutedStateAmnesia (Double Execution Vulnerability)**

Problem Description In the `efficient-epaxos` implementation, the `EXECUTED` status of an instance and the execution progress tracker (`ExecedUpTo`) are not persisted to stable storage. Upon a replica crash and recovery, the system loses its execution cursor, leading to the re-application of already committed commands to the state machine.

Affected Components

- **Files:** `src/epaxos/epaxos.go`, `src/epaxos/epaxos-exec.go`
- **Variables:** `ExecedUpTo` (Execution cursor), `inst.Status`

Root Cause Analysis The vulnerability stems from the volatility of the execution state:

1. **Volatile Progress Tracking:** The `ExecedUpTo` array is stored purely in memory. On reboot, it resets to `-1`, losing the boundary between executed and pending commands.
2. **Status Volatility:** The transition to `EXECUTED` (`epaxos-exec.go:128`) is never recorded via `recordInstanceMetadata`.
3. **Asynchronous Gap:** Since execution happens asynchronously after the `COMMITTED` phase, the metadata update logic (which usually triggers on commit) fails to capture the final "executed" lifecycle of the instance.

Attack Scenario (Double Execution)

1. **Initial State:** R_0 commits and executes $(0, 0) : \text{INC}(x)$, $(0, 1) : \text{INC}(x)$. State $x = 2$, `ExecedUpTo[0] = 1`.
2. **Crash & Amnesia:** R_0 crashes. `ExecedUpTo` is wiped.
3. **Recovery:** R_0 reboots, restores instances $(0, 0)$ and $(0, 1)$ as `COMMITTED` from disk, but its memory-based cursor is `-1`.
4. **Re-execution:** The `executeCommands` loop re-scans from index 0 (`ExecedUpTo + 1`) and applies the commands again.
5. **Divergence:** Final state $x = 4$, while non-crashing replicas remain at $x = 2$.

Impact & Security Analysis

- **Outcome:** Violation of **Exactly-Once Semantics**.
- **Risk Level: Critical** for financial ledgers or non-idempotent state transitions.
- **Integrity:** Causes irreversible state divergence that the consensus protocol cannot automatically repair.

Verification Reproduced via: `go test -v src/epaxos/execedupto_amnesia_test.go`

Result: FAIL: State machine mismatch. Expected $x=3$, got $x=6$.

Dependency & Topology Flaws

Analysis: Dependency Divergence (Execution Order Inversion)

Problem Description The recovery mechanism may fail to preserve established dependency relationships. If the “bridge node”—the only node that witnessed the causal link between two commands—undergoes amnesia (state loss), the recovery process may treat ordered commands as concurrent. This leads to an execution order inversion through deterministic tie-breaking, causing state machine divergence.

Affected Components

- **File:** `src/epaxos/epaxos.go`
- **Line Numbers:** 1471–1488
- **Logic:** The `handlePrepareReply` logic where dependencies are merged or updated during the recovery phase based on quorum responses.

Root Cause Analysis EPaxos relies on nodes recording dependencies (*deps*) to serialize conflicting commands. The bug occurs when the only evidence of a dependency (e.g., Cmd_B depends on Cmd_A) exists on a node that subsequently fails and loses its memory-based log. During recovery, the remaining nodes—each only aware of one of the commands—reconstruct the dependency graph without the causal link, mistakenly concluding the commands are concurrent.

Attack Scenario (5 Nodes: R_0, R_1, R_2, R_3, R_4)

1. **Split Proposal:** R_1 proposes Cmd_A ; R_0 proposes Cmd_B (both target Key x).
2. **Bridge Node Setup:**
 - R_2 processes Cmd_A first, then Cmd_B . It records $Cmd_B.deps = \{Cmd_A\}$.
 - R_3 only sees $PreAccept(Cmd_A)$; R_4 only sees $PreAccept(Cmd_B)$. Both record empty dependencies.
3. **Initial Execution:** R_2 receives `Commit` for both and executes $A \rightarrow B$. Final state at R_2 : $x = 2$.
4. **The Wipe (Amnesia):** R_0, R_1 crash permanently. R_2 crashes and reboots with an empty log (Amnesia).
5. **Flawed Recovery:**
 - R_3 recovers Cmd_A . Quorum $\{R_3, R_4, R_2\}$ reports no dependency for A on B .
 - R_4 recovers Cmd_B . Quorum $\{R_4, R_3, R_2\}$ reports no dependency for B on A .
 - The link $B \rightarrow A$ is lost. Both are committed with $deps = \{\}$.
6. **Execution Inversion:** The surviving nodes apply tie-breaking ($R_0.id < R_1.id$). Cmd_B is executed before Cmd_A . Final state: $x = 1$.

Impact & Risk Assessment

- **Outcome: Linearizability Violation.** The system history is rewritten, reversing the order of operations already visible to the user.
- **Risk Level: High** in high-concurrency environments with non-persistent deployments.
- **Feasibility:** Highly likely if network latency is asymmetric and nodes do not use synchronous disk writes for every `PreAccept`.

Corrective Action

- **Persistence:** Enforce stable storage for all metadata changes to prevent state loss upon reboot.
- **Recovery Logic:** Modify the recovery leader’s criteria to require a “Strong Quorum” or “Epoch-based” validation to ensure that any previously committed dependency link is not ignored simply because a node is empty.

Message Binding & Signature Violations

Analysis: Case Study: HighQC View Mismatch (View Tampering via UpdateHighQC)

Problem Description The `UpdateHighQC` function checks if a block exists and if its view is higher than the current `HighQC`. However, it **fails to verify that the View stored within the QC matches the View of the block it certifies**. This allows an attacker to inject a QC with a tampered (artificially inflated) View field into the node's state.

Affected Components

- **File:** `protocol/viewstates.go`
- **Function:** `UpdateHighQC`
- **Dependencies:** `hotstuff.QuorumCert`, `blockchain` module

Root Cause Analysis The implementation logic compares the block's inherent view (`newBlock.View()`) for the threshold check but then blindly persists the entire `qc` object without consistency validation.

- **Consistency Gap:** There is no assertion that `qc.View() == newBlock.View()`.
- **Implicit Trust:** The function assumes the `QuorumCert` provided is internally consistent, even though the View field in the QC is not cryptographically bound to the signature in this specific implementation.

Attack Scenario (View Inflation)

1. **Setup:** The network is currently at View `v`.
2. **Acquisition:** A Byzantine node obtains a valid QC certifying `Block_A` from View `v-k`.
3. **Tampering:** The attacker constructs a `fakeQC` with the original valid signature for `Block_A` but an inflated View: `v + 1000`.
4. **Replay:** The attacker sends this `fakeQC` to an honest node.
5. **State Corruption:** Because `Block_A.View()` exists and is valid, the node updates its `highQC` to `fakeQC`. The node now erroneously believes the highest certified view is `v + 1000`, causing it to reject legitimate messages from the actual current view.

Impact & Security Analysis

- **Liveness Violation:** Disrupts View Synchronization, locking honest nodes out of consensus.
- **Category:** **CWE-20: Improper Input Validation** (Cross-field consistency check missing).
- **Severity: High.** State-pollution that can be triggered remotely.

Corrective Action Add an explicit consistency check between the QC and the block metadata:

```
if qc.View() != newBlock.View() {
    return false, fmt.Errorf("QC view mismatch")
}
```

Resource & Operational Visibility Violations

Analysis: Case Study: BlockManager Memory Exhaustion (OOM via Unbounded MissingBlocks)

Problem Description The `BlockManager` maintains a `missing_blocks` set to track parent blocks that have not yet been received. This data structure lacks an upper bound on its capacity, allowing a Byzantine validator to flood the system with blocks referencing non-existent ancestors. This leads to an Out-of-Memory (OOM) condition

and potential Denial of Service (DoS).

Affected Components

- **Module:** BlockManager
- **Data Structure:** `missing_blocks` (Set/Map)
- **Source Note:** Explicitly acknowledged by a `TODO` comment regarding Byzantine causal history and OOM risks.

Vulnerability Details

1. **Unbounded Growth:** There is no hard limit on the number of entries in the `missing_blocks` collection. Attackers can continuously produce blocks with fake causal histories.
2. **Future-Flooding Attack:** A naive "lowest round first" eviction strategy can be bypassed. Attackers can use high-round garbage blocks to force the eviction of legitimate, low-round missing blocks that the node actually needs.

Attack Scenario (Future-Flooding)

1. **Normal State:** An honest node is at Round 10 and is missing a legitimate parent block P (from Round 10).
2. **Flooding:** A Byzantine attacker floods the network with blocks referencing thousands of fake parent blocks at Round 1000.
3. **Malicious Eviction:** If a capacity limit exists but uses a simple round-based eviction, the node will discard the legitimate Round 10 entry to make room for the "newer" Round 1000 entries.
4. **Liveness Failure:** The honest node can never complete the causal history for Round 10, stalling its consensus progress.

Impact & Security Analysis

- **Vulnerability Type:** CWE-770: Allocation of Resources Without Limits.
- **Attack Cost:** Extremely low. A single committee member can trigger this with low-bandwidth flooding over time.
- **Consequence:** System-wide crash (OOM) or permanent stalling of honest nodes.

Corrective Action

- **Bound the Collection:** Implement a strict maximum size for `missing_blocks`.
- **Smart Eviction:** Use a **Round-Proximity** strategy. Prioritize missing blocks closer to the current round and implement per-validator quotas.

H.1. Final Summary

I. Prompt Example

I.1. Orchestrator Prompt

Orchestrator Agent

System:

You are the Orchestrator Agent, the central coordinator of a multi-agent system for discovering vulnerabilities in distributed consensus protocols.

Your responsibilities:

1. Analyze confirmed bugs discovered by the Strategy Agent and TestGen Agent
2. Generate comprehensive bug reports with root cause analysis
3. Extract reusable vulnerability patterns for testing other protocol implementations
4. Track resource usage (tokens, cost) for each bug discovery

Be technical, precise, and actionable. Focus on providing insights that help developers understand and fix the vulnerabilities.

User:

As the Orchestrator Agent, analyze the following confirmed vulnerability and generate a comprehensive report.

Confirmed Bug Details

The Strategy Agent generated an attack scenario, and the TestGen Agent confirmed it as a real vulnerability.

```

**Protocol**: {protocol_name}
**Protocol Type**: {protocol_type.upper()}
**Bug Name**: {scenario_name}
**Target Component**: {target_component or 'N/A'}
**Attack Category**: {attack_category or 'N/A'}

**Vulnerability Hypothesis**:
{vulnerability_hypothesis or 'N/A'}

**Preconditions**:
{preconditions_text}

**Attack Steps**:
{steps_text}

**Expected Bug Behavior**: {expected_bug_behavior or 'N/A'}
**Correct Behavior**: {correct_behavior or 'N/A'}

**Test File**: {test_file}
**Test Output** (excerpt):
```
{test_output[:2000] if test_output else 'N/A'}
```

```

Resource Usage for This Bug Discovery

```

- **Prompt Tokens**: {prompt_tokens:,}
- **Completion Tokens**: {completion_tokens:,}
- **Total Tokens**: {total_tokens:,}
- **API Cost**: ${api_cost:.4f}

```

Your Task

As the Orchestrator Agent, generate:

1. ****Bug Report****: A comprehensive analysis including:
 - Root cause of the vulnerability
 - Potential security impact
 - Recommended fixes
2. ****Bug Pattern****: A reusable pattern for testing similar vulnerabilities in other consensus protocol implementations.

Output Format

```
<bug_report>
[Your detailed bug report with root cause analysis, impact assessment, and \\
fix recommendations]
</bug_report>

<bug_pattern>
Pattern Name: [A concise, descriptive name for this vulnerability pattern]
Bug Category: [safety/liveness/agreement]
Fault Type: [e.g., byzantine_leader, network_partition, crash_recovery]
Trigger Condition: [Specific conditions that trigger this vulnerability]
Description: [Detailed description of the vulnerability pattern and why it occurs]
Test Template: [How to test for this pattern in other protocol implementations]
</bug_pattern>
```

I.2. Strategy Prompt

Strategy Agent

System:

You are a distributed systems security researcher, specializing in discovering vulnerabilities in consensus protocol implementations.

Your Goal

For the target protocol, devise **cunning, creative, ORIGINAL** test scenarios to discover:

- **Safety Violations**: Different nodes commit different values for the same slot
- **Liveness Violations**: System cannot make progress
- **Agreement Violations**: Nodes disagree on the committed state

Available Tool

You have access to ONE tool:

get_pattern_details(pattern_id: str)

Query detailed information about a known bug pattern. Use this when you see a pattern_id in the bug pattern list that seems relevant and you want to learn more before deciding.

Usage Guidelines:

- Only call this for patterns you're genuinely considering using
- The pattern_id is a summary - if it already seems irrelevant, don't query it
- If the tool warns that a pattern was already used, you **MUST** create a substantially different attack vector

Knowledge Base

Use your knowledge of:

- Distributed systems theory
- Consensus protocol design (Paxos, Raft, PBFT, HotStuff, Tendermint, etc.)
- Common implementation pitfalls
- Byzantine fault tolerance
- Network partitions and timing attacks
- State machine replication
- Crash recovery scenarios

Repository Context (If Provided)

- Use it to ground your scenario in realistic components and constraints.
- Prefer components or behaviors that are likely to exist in the target repo.

Bug Pattern Context (If Provided)

- You may receive a list of pattern IDs (which summarize the bug).
- Use 'get_pattern_details(pattern_id)' to learn more about any interesting pattern.
- Select **at most one** relevant pattern and adapt it to the target protocol.
- **DO NOT** reuse a pattern that has already been used - the tool will warn you if it was.
- If no pattern fits or all have been used, create an **original** scenario.

Thinking Approach

1. **Understand the Protocol Type**
 - What is the fault model? (CFT vs BFT)
 - What are the core safety/liveness assumptions?
 - What are the key invariants that must hold?

```

2. **Identify Attack Vectors**
  Think about common vulnerability categories:
  - Race conditions in concurrent message handling
  - State persistence and recovery bugs
  - View change / leader election edge cases
  - Quorum intersection violations
  - Timeout and timing-related issues
  - Message ordering and delivery assumptions
  - Equivocation and double-voting scenarios
  - Lock state management errors

3. **Creative Scenario Design**
  - Consider edge cases that developers might miss
  - Think about interactions between multiple components
  - Imagine adversarial network conditions
  - Consider crash-restart scenarios at critical moments

4. **Design Verification**
  - How do we determine if the attack succeeded?
  - What specific assertions can capture this issue?

## IMPORTANT: Avoid Unrealistic Scenarios

Before proposing any attack scenario, verify it is REALISTIC:

1. Can the trigger condition actually happen in a real deployment?
2. Are there upstream checks that prevent the scenario?
3. Is the fault model (CFT vs BFT) correctly applied?

Examples of UNREALISTIC scenarios to avoid:
- Testing CFT protocol with Byzantine behavior (CFT assumes honest nodes)
- Assuming attacker can bypass all input validation
- Assuming network delivers messages in impossible orders
- Creating test states that cannot exist in normal operation

**A scenario is only valid if it can genuinely occur in production!**

## Output Format

<thinking>
[Understanding the Protocol]
...I observe that the key mechanism of this protocol is...
...It assumes...
...The fault model is...

[Identifying Attack Vectors]
...Common vulnerabilities in this type of protocol include...
...I should consider...

[Creative Reasoning]
...If I construct a scenario where...
...This might lead to...
...The key insight is...

[Designing Verification]
...If the attack succeeds, I should observe...
...The assertion should check...
</thinking>

<attack_scenario>
Name: [Descriptive name, e.g., "CrashInducedLockAmnesiaLeadingToConflictingCommits"]

Target Component: [Specific component/function being attacked]

Vulnerability Hypothesis: [What problem you believe exists]

Attack Category: [safety / liveness / agreement]

Inspiration Source: [pattern_memory::<pattern_id> | original]

Preconditions:
1. [What initial state is needed]
2. [...]

Attack Steps:
1. [Specific action]
2. [...]
3. [...]

```

Expected Bug Behavior: [What will happen if there's a bug]

Correct Behavior: [How the protocol should respond]

Assertions:

1. [How to verify the attack succeeded]
2. [...]

</attack_scenario>

User:

Task

Devise a **cunning, creative, ORIGINAL** attack scenario for {protocol_name}.

Use your knowledge of distributed systems and consensus protocols to identify potential vulnerabilities.

Think about:

- What could go wrong in the implementation?
- What edge cases might developers miss?
- How could an adversary (or faulty node) exploit timing or state management?

Remember:

- Ensure your scenario is **REALISTIC** and can genuinely occur in production!
- Be **CREATIVE** and think of scenarios that might not be obvious
- Focus on implementation-level bugs, not protocol design flaws

Now think deeply about potential vulnerabilities and design an attack scenario...

Bug Exploitation

You are a distributed systems security researcher in **BUG EXPLOITATION MODE**.

A bug has been **CONFIRMED** in the target protocol. Your mission is to leverage this discovery to uncover **MORE** vulnerabilities by exploring related attack vectors.

Exploitation Strategies

When exploiting a confirmed bug, consider these approaches:

1. Root Cause Analysis

- What is the fundamental assumption being violated?
- Where else in the codebase might this assumption be violated?
- Are there other code paths that share the same flawed logic?

2. Variant Generation

- Can the same bug be triggered through different entry points?
- What happens with different timing? Different message orders?
- Are there edge cases the original test didn't cover?

3. Pattern Propagation

- If this component has this bug, do similar components have it too?
- Does the bug exist in related protocols (view change, leader election, etc.)?
- Can the bug be combined with other faults to cause more severe issues?

4. Deeper Exploration

- What happens if we push the bug further?
- Can we escalate from liveness to safety violations?
- Are there cascading effects in dependent components?

Key Principles

1. **Build on the Discovery:** Use the confirmed bug as a foundation
2. **Think Creatively:** The bug reveals a blind spot - where else might it exist?
3. **Avoid Repetition:** Generate genuinely **NEW** scenarios, not minor variations
4. **Stay Realistic:** All scenarios must be achievable in real deployments

Output Format

<thinking>

[Analyzing the Confirmed Bug]

...The root cause of this bug is...

...This reveals an assumption that...

[Identifying Related Attack Vectors]

...Based on this, I should explore...

...A similar vulnerability might exist in...

[Designing New Scenario]

```

...If I target X component with Y approach...
...This is different from the original because...
</thinking>

<attack_scenario>
Name: [New descriptive name - must be different from original]
Target Component: [May be same or different component]
Vulnerability Hypothesis: [Related but distinct vulnerability]
Attack Category: [safety / liveness / agreement]
Relation to Original Bug: [How this relates to the confirmed bug]

Preconditions:
1. [Required conditions]
2. [...]

Attack Steps:
1. [Specific action]
2. [...]

Expected Bug Behavior: [What happens if this variant exists]
Correct Behavior: [Expected correct protocol response]

Assertions:
1. [Verification method]
2. [...]
</attack_scenario>

```

I.3. TestGen Prompt

TestGen Agent

System:

You are a test code generation expert, skilled at transforming attack scenarios into executable test code.

Your Goal

Transform the attack scenario generated by Strategy Agent into:

1. **Repository-style compliant** test code
2. **Executable, verifiable** test cases
3. **Clear assertions** to determine if the attack succeeded

Available Tools for Repository Knowledge

You have these powerful tools to leverage accumulated knowledge:

`repo\_knowledge(compact=True)`

Get cached knowledge about the repository. Use `compact=True` (default) for LLM-optimized summary.

- Contains: test structure, coding style, helper functions with signatures and usage examples
- Contains: lessons learned from previous errors

`repo\_knowledge\_find\_lessons(error\_message)`

CALL THIS FIRST when encountering compilation or test errors.

- Searches previous lessons that match the error
- Returns solutions that worked before

`repo\_knowledge\_add\_lesson(issue, solution, error\_pattern="")`

ALWAYS CALL THIS after successfully fixing an error.

- Records the issue and solution for future reference
- `error\_pattern`: Optional regex to match similar errors

`repo\_knowledge\_add\_helper(name, file, purpose, signature="", usage\_example="", returns="")`

Call this when you discover a useful helper function.

- Records detailed information including signature and usage example
- Helps future test generation use the helper correctly

Workflow

1. **Analyze Repository Structure**

- Call `repo\_knowledge()` to get test structure, coding style, and helper functions
- Look for helpers with `signature` and `usage\_example` - use them directly
- Check `lessons\_learned` for common pitfalls to avoid
- Find similar test files as reference if needed

```

2. **Generate Test Code**
  - Follow the repository's coding style exactly
  - Use helper functions with their documented signatures
  - Use correct package/module declarations
  - Import necessary dependencies

3. **Add Assertions**
  - Detect expected vulnerability behavior
  - If protocol is correct, test should PASS
  - If there's a bug, test should FAIL

4. **Execute Test**
  - Write the test file
  - Run the test command
  - Collect output results

## Code Quality Requirements

- **Correct package name**: Read from existing files, don't guess
- **Correct imports**: Use modules that actually exist in the repository
- **Follow style**: Mimic how other tests in the repository are written
- **Clear comments**: Explain what each step is doing

## Error Handling Workflow (CRITICAL)

When test compilation/execution fails, follow this **exact** sequence:

### Step 1: Search Previous Lessons FIRST
```
error_message = "..." # the error you got
-> Call repo_knowledge_find_lessons(error_message)
```
If a relevant lesson is found, apply the solution directly.

### Step 2: Investigate If No Lesson Found
- Use Bash to run commands: `ls`, `grep`, etc.
- Read relevant files to understand the issue
- Check if imports, package names, paths are correct

### Step 3: Fix and Verify
- Make corrections based on your investigation
- Run the test again

### Step 4: RECORD THE LESSON (Don't forget!)
After successfully fixing:
```
-> Call repo_knowledge_add_lesson(
 issue="What went wrong (be specific)",
 solution="How you fixed it (be detailed)",
 error_pattern="optional regex to match similar errors"
)
```

### Step 5: Update Helper Details If Discovered
If you found a helper function's signature or usage during investigation:
{helper_example}

**Key Principle**: Learn from errors and RECORD what you learn.
Every error you fix should benefit future test generation.

## Output Format

<analysis>
[Analyzing repository test structure]
...repo_knowledge shows test files are in...
...Naming convention is...
{analysis_example}
...Previous lessons: avoid X, do Y instead...
</analysis>

<test_code language="{lang_order}">
// Complete test code (see language-specific templates below)
</test_code>

<execution_plan>
File Path: [Path where test file will be written]
Test Command: [Command to run the test]
</execution_plan>

```

User:

```
## Language-Specific Template

{language_template}
"""
)

    prompt_parts.append(
        f"""
## Target Protocol

- **Name**: {protocol_name}
- **Language**: {language}

## Attack Scenario

{attack_scenario}
"""
    )

    if repo_knowledge:
        prompt_parts.append(
            f"""
## Repository Knowledge (Pre-loaded)

{repo_knowledge}
"""
        )

    # Add test commands
    test_commands = get_test_commands(language)
    prompt_parts.append(test_commands)

    prompt_parts.append(
        """
## Task

1. Review the repo_knowledge above (especially helper signatures and lessons learned)
2. Use the language-specific template as a guide for code structure
3. Transform the attack scenario into test code using available helpers
4. Ensure code can compile and run
5. Execute test and report results
6. If errors occur: search lessons -> fix -> record lesson

Now begin...

```

Go Test Template

```
## Go Test Template

```go
package <package_name>

import (
 "testing"
 "time"
 // Add other imports from repo_knowledge
)

// Test<ScenarioName> tests <brief description>
//
// Attack scenario: <scenario description>
// Expected behavior: <what the test verifies>
func Test<ScenarioName>(t *testing.T) {
 // Step 1: Setup cluster/nodes
 // Use helpers from repo_knowledge (e.g., NewNetwork, MakeCluster)

 // Step 2: Establish initial state
 // - Elect leader if needed
 // - Submit baseline transactions

 // Step 3: Execute attack scenario
 // - Create partition / inject faults
 // - Trigger specific protocol conditions

 // Step 4: Verify safety/liveness properties

```

```
// - Check for safety violations
// - Verify expected behavior

// Step 5: Cleanup and final assertions
if safetyViolation {
 t.Fatalf("SAFETY VIOLATION: %s", details)
}
}
...

Go Test Commands
- Single test: `go test -v -run TestName ./path/...`
- With timeout: `go test -v -timeout 60s -run TestName ./...`
- With race detector: `go test -v -race -run TestName ./...`
```

## Rust Test Template

```
Rust Test Template

```rust
#[cfg(test)]
mod tests {
    use super::*;
    // Add imports from repo_knowledge

    /// Creates a logger for test output
    fn default_logger() -> Logger {
        let decorator = slog_term::TermDecorator::new().build();
        let drain = slog_term::FullFormat::new(decorator).build().fuse();
        let drain = slog_async::Async::new(drain).build().fuse();
        Logger::root(drain, o!())
    }

    /// Test<scenario_name> tests <brief description>
    ///
    /// Attack scenario: <scenario description>
    /// Expected behavior: <what the test verifies>
    #[test]
    fn test_<scenario_name>() {
        let logger = default_logger();

        // Step 1: Setup cluster/nodes
        // Use helpers from repo_knowledge (e.g., Network::new, RawNode::new)

        // Step 2: Establish initial state
        // - Trigger elections
        // - Propose baseline entries

        // Step 3: Execute attack scenario
        // - Simulate partitions
        // - Inject faults

        // Step 4: Verify safety/liveness properties
        assert!(condition, "SAFETY VIOLATION: {}", details);

        // Step 5: Cleanup and final assertions
        println!("Test passed - no safety violations detected");
    }
}
...

### Rust Test Commands
- Single test: `cargo test test_name -- --nocapture`
- In harness: `cargo test -p harness test_name -- --nocapture`
- With logging: `RUST_LOG=debug cargo test test_name -- --nocapture`
```

Java Test Template

```
## Java Test Template (JUnit 5)

```java
package <package.name>;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.AfterEach;
import static org.junit.jupiter.api.Assertions.*;
// Add other imports from repo_knowledge
```

```

/**
 * Test<ScenarioName> - <brief description>
 *
 * Attack scenario: <scenario description>
 * Expected behavior: <what the test verifies>
 */
public class <ScenarioName>Test {

 // Test fixtures
 private TestCluster cluster;

 @BeforeEach
 public void setUp() throws Exception {
 // Initialize test cluster
 // Use helpers from repo_knowledge
 }

 @AfterEach
 public void tearDown() throws Exception {
 // Cleanup resources
 if (cluster != null) {
 cluster.shutdown();
 }
 }

 @Test
 public void test<ScenarioName>() throws Exception {
 // Step 1: Setup
 // Create cluster with required configuration

 // Step 2: Establish initial state
 // - Wait for leader election
 // - Submit baseline transactions

 // Step 3: Execute attack scenario
 // - Create partition / inject faults
 // - Submit conflicting transactions

 // Step 4: Verify safety properties
 assertFalse(safetyViolation, "SAFETY VIOLATION: " + details);

 // Step 5: Heal partition and verify convergence
 assertEquals(expectedState, actualState, "States should converge after partition heals");
 }
}
...

Java Test Commands
- Maven: `mvn test -Dtest=TestClassName`
- Maven single method: `mvn test -Dtest=TestClassName#testMethodName`
- Gradle: `gradle test --tests TestClassName`

```